МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«БРЕСТКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ
Кафедра ИИТ

Отчет по лабораторной работе №3

Специальность ПО-12

Выполнил:
Студент 3 курса
Группы ПО-12
Мельник Р.С.
Проверила:
Кулик А. Д..

Брест 2026

Цель работы: приобрести навыки применения паттернов проектирования

при решении практических задач с использованием языка Python

Задание 1

Вариант: Проект «Бургер-закусочная». Реализовать возможность формирования заказа из определенных позиций (тип бургера (веганский, куриный и т.д.)), напиток (холодный – пепси, кока-кола и т.д.; горячий – кофе, чай и т.д.), тип упаковки – с собой, на месте. Должна формироваться итоговая стоимость заказа.

Код программы:

```
# pylint: disable=invalid-name
"""
Модуль реализует систему заказа в бургерной с использованием паттерна Строитель.
"""



class Order:
    """Класс, представляющий готовый заказ клиента."""


    def __init__(self):
        """Инициализация пустого заказа."""
        self.items = []
        self.packaging = ""
        self.total_price = 0.0
```

```
    def add_item(self, name: str, price: float):
        """Добавление позиции в заказ."""
        self.items.append(name)
        self.total_price += price

    def set_packaging(self, pack_type: str, price: float):
        """Установка типа упаковки."""
        self.packaging = pack_type
        self.total_price += price

    def __str__(self):
        """Строковое представление заказа."""
        items_str = ", ".join(self.items)
        return (
            f"--- ВАШ ЗАКАЗ ---\n"
            f"Состав: {items_str}\n"
            f"Упаковка: {self.packaging}\n"
            f"Итоговая стоимость: {self.total_price:.2f} руб.\n"
            f"-----------------"
        )


class OrderBuilder:
    """Абстрактный класс строителя заказов."""

    def reset(self):
```

```
        """Сброс состояния строителя."""
        raise NotImplementedError

    def add_burger(self, burger_type: str):
        """Метод добавления бургера."""
        raise NotImplementedError

    def add_drink(self, drink_name: str):
        """Метод добавления напитка."""
        raise NotImplementedError

    def set_packaging(self, to_go: bool):
        """Метод выбора упаковки."""
        raise NotImplementedError

    def get_result(self) -> Order:
        """Получение готового объекта заказа."""
        raise NotImplementedError


class BurgerOrderBuilder(OrderBuilder):
    """Конкретная реализация строителя для бургер-закусочной."""

    def __init__(self):
        """Инициализация строителя и прейскуранта цен."""
        self._order = Order()
```

```
        self._prices = {
            "веганский": 250.0,
            "куриный": 300.0,
            "говяжий": 350.0,
            "пепси": 100.0,
            "кока-кола": 100.0,
            "кофе": 150.0,
            "чай": 80.0,
            "с собой": 20.0,
            "на месте": 0.0,
        }

    def reset(self):
        """Создание нового чистого объекта заказа."""
        self._order = Order()

    def add_burger(self, burger_type: str):
        """Добавление бургера в заказ."""
        name = f"Бургер ({burger_type})"
        price = self._prices.get(burger_type.lower(), 0.0)
        self._order.add_item(name, price)
        return self

    def add_drink(self, drink_name: str):
        """Добавление напитка в заказ."""
        name = f"Напиток ({drink_name})"
```

```
        price = self._prices.get(drink_name.lower(), 0.0)
        self._order.add_item(name, price)
        return self

    def set_packaging(self, to_go: bool):
        """Выбор типа упаковки (на месте или с собой)."""
        if to_go:
            self._order.set_packaging("С собой", self._prices["с собой"])
        else:
            self._order.set_packaging("На месте", self._prices["на месте"])
        return self

    def get_result(self) -> Order:
        """Возврат собранного заказа и подготовка к следующему."""
        result = self._order
        self.reset()
        return result


class Waiter:
    """Класс Директора (Официанта), управляющий процессом сборки."""

    def __init__(self):
        """Инициализация официанта без назначенного строителя."""
        self._builder = None
```

```
    @property
    def builder(self) -> OrderBuilder:
        """Свойство для получения текущего строителя."""
        return self._builder

    @builder.setter
    def builder(self, value: OrderBuilder):
        """Свойство для установки строителя."""
        self._builder = value

    def construct_vegan_combo(self):
        """Сборка стандартного веганского набора."""
        self._builder.add_burger("веганский")
        self._builder.add_drink("чай")
        self._builder.set_packaging(False)

    def construct_meat_to_go(self):
        """Сборка мясного набора на вынос."""
        self._builder.add_burger("говяжий")
        self._builder.add_drink("кока-кола")
        self._builder.set_packaging(True)


if __name__ == "__main__":
    burger_builder = BurgerOrderBuilder()
```

```
    waiter_service = Waiter()
    waiter_service.builder = burger_builder

    print("--- Демонстрация работы Строителя ---\n")

    print("Заказ №1 (Веган-комбо через Директора):")
    waiter_service.construct_vegan_combo()
    print(burger_builder.get_result())

    print("\nЗаказ №2 (Индивидуальный заказ напрямую через Строителя):")
burger_builder.add_burger("куриный").add_drink("кофе").set_packaging(True)
    print(burger_builder.get_result())

    print("\nЗаказ №3 (Мясной набор с собой):")
    waiter_service.construct_meat_to_go()
    print(burger_builder.get_result())
```

Результат:

```
Итоговая стоимость: 470.00 руб.
-----------------
```

Задание 2

Вариант:Проект «IT-компания». В проекте должен быть реализован класс «Сотрудник» с субординацией (т.е. должна быть возможность определения кому подчиняется сотрудник и кто находится в его подчинении). Для каждого сотрудника помимо сведений о субординации хранятся другие данные (ФИО, отдел, должность, зарплата). Предусмотреть возможность удаления и добавления сотрудника.

Код программы:

```
# pylint: disable=invalid-name
"""
Модуль для управления структурой IT-компании.
Реализует иерархию сотрудников через паттерн Компоновщик.
"""
class Employee:
    """Класс сотрудника, поддерживающий древовидную структуру подчинения."""

    def __init__(self, name: str, department: str, position: str, salary: float):
        """Инициализация данных сотрудника."""
        self.name: str = name
        self.department: str = department
        self.position: str = position
        self.salary: float = salary
        self._subordinates: list = []
        self.manager: Employee = None

    def add_subordinate(self, employee: "Employee") -> None:
        """Добавляет другого сотрудника в список подчиненных."""
```

```
    if employee not in self._subordinates:
        employee.manager = self
        self._subordinates.append(employee)
        print(f"-> {employee.name} теперь подчиняется {self.name}")

def remove_subordinate(self, employee: "Employee") -> None:
    """Удаляет сотрудника из списка подчиненных."""
    if employee in self._subordinates:
        employee.manager = None
        self._subordinates.remove(employee)
        print(f"-> {employee.name} удален из подчинения {self.name}")
    else:
        print(f"-> Ошибка: {employee.name} не подчиняется {self.name}")

def get_subordinates(self) -> list:
    """Возвращает список всех прямых подчиненных."""
    return self._subordinates

def show_details(self, level: int = 0) -> None:
    """Рекурсивно выводит структуру подчинения сотрудника."""
    indent = "  " * level
    m_name = self.manager.name if self.manager else "Никто (Топ-менеджер)"
    print(f"{indent}□ {self.name} [{self.position}]")
    print(f"{indent}   Отдел: {self.department}, Зарплата: {self.salary}")
    print(f"{indent}   Подчиняется: {m_name}")
```

```
        if self._subordinates:
            print(f"{indent}   Подчиненные ({len(self._subordinates)}):")
            for sub in self._subordinates:
                sub.show_details(level + 2)

    def __str__(self) -> str:
        """Возвращает краткую информацию о сотруднике в виде строки."""
        return f"{self.name} ({self.position})"



if __name__ == "__main__":
    print("--- Формирование структуры IT-компании ---\n")

    main_ceo = Employee("Иван Иванов", "Управление", "Генеральный
директор", 500000)
    tech_cto = Employee("Сергей Петров", "IT", "Технический директор",
350000)
    hr_director = Employee("Анна Сидорова", "HR", "Глава HR", 200000)

    main_ceo.add_subordinate(tech_cto)
    main_ceo.add_subordinate(hr_director)

    lead = Employee("Алексей Сорокин", "IT", "Team Lead", 250000)
    junior = Employee("Дмитрий Лукин", "IT", "Junior Developer", 80000)
    hr_recruiter = Employee("Елена Кравц", "HR", "Рекрутер", 90000)

    tech_cto.add_subordinate(lead)
```

```
lead.add_subordinate(junior)
hr_director.add_subordinate(hr_recruiter)

print("\n--- Полная иерархия компании ---")
main_ceo.show_details()

print("\n--- Проверка удаления из структуры ---")
lead.remove_subordinate(junior)

print("\n--- Иерархия отдела IT после изменений ---")
tech_cto.show_details()
```

Результат:

```
        Отдел: IT, Зарплата: 250000
        Подчиняется: Сергей Петров
```

Задание 3

Вариант:Проект «Расчет зарплаты». Для задания, указанного во втором пункте («IT-компания») реализовать расчет зарплаты с выводом полного отчета. Порядок вывода сотрудников в отчете – по старшинству для каждого отдела.

Код программы:

```
# pylint: disable=invalid-name, too-many-arguments, too-many-positional-arguments, too-few-public-methods
"""
Модуль для расчета зарплаты сотрудников IT-компании.
Реализует формирование отчета через паттерн Посетитель.
"""

from typing import List, Dict


class Employee:
    """Класс, представляющий сотрудника как элемент для посещения."""

    def __init__(self, name: str, dept: str, pos: str, sal: float, level: int):
        """Инициализация данных сотрудника и его уровня в иерархии."""
        self.name: str = name
        self.department: str = dept
        self.position: str = pos
        self.salary: float = sal
        self.level: int = level
        self.subordinates: List['Employee'] = []

    def add_subordinate(self, emp: 'Employee') -> None:
```

```
        """Добавляет сотрудника в список подчиненных."""
        self.subordinates.append(emp)

    def accept(self, visitor: 'CompanyVisitor') -> None:
        """Принимает посетителя и передает его подчиненным."""
        visitor.visit_employee(self)
        for sub in self.subordinates:
            sub.accept(visitor)


class CompanyVisitor:
    """Абстрактный базовый класс для посетителей компании."""

    def visit_employee(self, employee: Employee) -> None:
        """Метод для обработки данных конкретного сотрудника."""
        raise NotImplementedError


class SalaryReportVisitor(CompanyVisitor):
    """Посетитель для генерации иерархического отчета по зарплате."""

    def __init__(self):
        """Инициализация хранилища данных по отделам."""
        super().__init__()
        self.data: Dict[str, List[Employee]] = {}
```

```
def visit_employee(self, employee: Employee) -> None:
    """Собирает сотрудников в группы по их отделам."""
    if employee.department not in self.data:
        self.data[employee.department] = []
    self.data[employee.department].append(employee)

def print_report(self) -> None:
    """Выводит отчет, отсортированный по отделам и старшинству."""
    header = (f"{'ОТДЕЛ':<15} | {'СТАТУС':<5} | {'ДОЛЖНОСТЬ':<20} | "
              f"{'ФИО':<18} | {'ЗАРПЛАТА':<10}")
    print(header)
    print("-" * 80)

    total_company_salary = 0.0

    for dept in sorted(self.data.keys()):
        # Сортировка по уровню (старшинству)
        employees = sorted(self.data[dept], key=lambda x: x.level)
        dept_total = 0.0

        for emp in employees:
            print(f"{emp.department:<15} | Lvl {emp.level} | "
                  f"{emp.position:<20} | {emp.name:<18} | {emp.salary:<10.2f}")
            dept_total += emp.salary

        print(f"{' ' * 45} Итого по отделу {dept}: {dept_total:.2f} руб.")
```

```
        print("-" * 80)
        total_company_salary += dept_total

    print(f"ОБЩИЙ ФОНД ОПЛАТЫ ТРУДА КОМПАНИИ: {total_company_salary:.2f} руб.")



if __name__ == "__main__":
    # Инициализация структуры
    top_ceo = Employee("Иван Иванов", "Управление", "Ген. директор", 500000, 0)

    tech_dir = Employee("Сергей Петров", "IT", "Тех. директор", 350000, 1)
    hr_dir = Employee("Анна Сидорова", "HR", "Глава HR", 200000, 1)

    lead_dev = Employee("Алексей Сорокин", "IT", "Team Lead", 250000, 2)
    junior_dev = Employee("Дмитрий Лукин", "IT", "Junior Dev", 80000, 3)
    recruiter = Employee("Елена Кравц", "HR", "Рекрутер", 90000, 2)

    # Построение дерева
    top_ceo.add_subordinate(tech_dir)
    top_ceo.add_subordinate(hr_dir)
    tech_dir.add_subordinate(lead_dev)
    lead_dev.add_subordinate(junior_dev)
    hr_dir.add_subordinate(recruiter)

    # Генерация отчета
```

```
salary_visitor = SalaryReportVisitor()
print("Генерация отчета по зарплате...\n")
top_ceo.accept(salary_visitor)
salary_visitor.print_report()
```

Результат:

```
ОБЩИЙ ФОНД ОПЛАТЫ ТРУДА КОМПАНИИ: 1470000.00 руб.
```

Вывод: приобрели навыки применения паттернов проектирования при решении практических задач с использованием языка Python